New A000104 terms: algorithms and program

Charles (Chaz) Reid

November 2, 2026

Abstract

A000104 counts the free polyominoes of n cells that have no holes. The OEIS marks it *hard*, and its b-file ends at $n = 40$. This manuscript gives $a(41)$ to $a(46)$, A001419 at the same n, and A006724 at $n = 43$ to 46, and shows that the published value of A006724 at $n = 42$ is 54,214 too small. Burnside's lemma reduces each term to five counts. A transfer matrix gives the largest, the polygons counted by area, and a new test at each polygon's center computes the count with 180-degree symmetry inside the same sweep; a search over the orbits of each symmetry gives the other symmetric counts. The program, in Rust, splits the sweep's states across threads by a hash of each state, counts modulo two 64-bit moduli and rebuilds the exact values from the residues, streams its states through disk when they outgrow memory, and resumes from checkpoints. The sweep's work grows by a factor of about 1.669 a term and its memory by 1.578, in alternating steps: each odd n adds a new widest strip that the even n after it reuses, so the work rises about 2.75 times for each pair of terms. Further terms depend on lowering the growth rate, and this part lists the changes that would.

1 Introduction

The OEIS labels A000104 *hard*: new terms are computationally difficult and expensive. The sequence counts the polyominoes of n cells that have no holes, up to translation, rotation and reflection, where a polyomino is a finite set of cells of the square grid connected through shared edges. Its b-file, by John Mason, ends at $n = 40$. This manuscript extends it to $n = 46$ and describes the algorithms and the program that computed the terms.

Burnside's lemma reduces $a(n)$ to five counts of polyominoes fixed by the symmetries of the square (Section 2). The largest of the five is the number $F(n)$ of hole-free polyominoes counted up to translation alone. This is A006724, the number of self-avoiding polygons of area n on the square lattice. Its b-file, by Iwan Jensen, ends at $n = 42$; according to Jensen and Guttmann (2000, p. 4), the 42-term series is the one given by Guttmann et al. (2000). The other four counts are of symmetric polyominoes, which are far fewer: at $n = 46$ the largest of them is about 3×10^{-13} of $F(46)$.

The new terms $a(41), \ldots, a(46)$ also give A001419, the polyominoes with holes, at the same n: A001419 = A000105 − A000104, and A000105 is known to $n = 59$ (Shirakawa 2025). We also compute $F(43)$ to $F(46)$, and show that the value of $F(42)$ in the A006724 b-file is 54,214 too small. At $n = 41$, polyominoes with holes outnumber those without for the first time. Section 5 gives the terms and the checks they pass.

From $n = 42$ on, each term needs a new value of $F(n)$, which we compute with a transfer matrix (Section 3), and its cost grows geometrically. A run to N gives every term up to N but cannot be extended past N, so the cost that matters is that of a run, and the N it stops at is a choice. Write the work of a run as $c\,g^n$: faster code and smaller states lower the constant c, and better pruning lowers the growth rate g. Fitted to the laptop's runs to $n = 42$, the transfer matrix's work grows by 1.669 a term and its peak memory by 1.578, and the search that computes the symmetric counts by 1.982; on one processor the runs from $n = 43$ to 46 put the search at about 1.89. The growth alternates: each odd N adds the transfer matrix's widest strip, about half its work, and the even N after it reuses that strip, so the work rises about 2.75 times for each pair of terms and the runs worth making stop at an even N (Section 6).

Memory sets the sharpest limit: the peak memory grows by 1.578 a term, and a sweep that outgrows one machine's memory must stream its states through disk. Constant-factor savings in memory, such as 16-bit counts or streaming the states through disk, move that limit by a few terms; a lower growth rate g moves it further with every term. Section 7 separates the two, and Section 8 lists the work that would lower g.

The methods here combine two lines of earlier work. Transfer matrices count the fixed polyominoes, A001168: Barequet and Ben-Shachar (2026) extended that sequence from 56 terms to 70 with a transfer matrix on a lattice rotated by 45 degrees, in about 15,000 CPU hours. For free polyominoes, Shirakawa (2025) reached $n = 59$ with the same Burnside reduction, counting the mirror-symmetric classes with a transfer matrix on one side of the axis and the rotationally symmetric ones by search. We use a transfer matrix for F and for the half-turn count R_2, and Redelmeier's search (Redelmeier 1981) for all the symmetric counts. The count R_2 comes from the same sweep as F, with a test at each polygon's center and no sweep of its own (Section 3.2); Shirakawa (2025, Sect. 6.1) suggests a transfer matrix for point-symmetric polyominoes as future work. Transfer matrices for self-avoiding polygons go back to Enting (1980), and Jensen and Guttmann (2000) describe their enumeration of polygons by area as based on the finite-lattice method of Conway (1995) and similar to Enting's. The search's forced cores follow Mason's counters for this sequence, which grow each symmetry class from its center squares by the same add-or-exclude recursion (Mason 2022).

Section 2 sets out the reduction, Section 3 the algorithms, Section 4 the program, and Section 5 the results. Section 6 measures how the work and memory grow, Section 7 separates the constant from the growth rate, and Section 8 lists further work.

2 Counting by symmetry

2.1 Definitions

Cells are the unit squares of the grid, and two cells are *adjacent* if they share an edge. A *fixed* n-omino is a set of n cells, connected under adjacency, counted up to translation. A *free* n-omino is counted up to the eight symmetries of the square: the rotations by 0, 90, 180 and 270 degrees, and the reflections in two axis-parallel lines and two diagonal lines. A *hole* of a polyomino is a finite set of empty cells that is connected under adjacency and is maximal with that property. Two empty cells that touch only at a corner are not adjacent, so they can lie in different holes, or one in a hole and one outside; the smallest polyomino with a hole has

seven cells (Figure 1). A *self-avoiding polygon* is a closed path along grid edges that visits no vertex twice, and its *area* is the number of cells it encloses. Where convenient we identify a polyomino with the union of its closed cells, a subset of the plane.

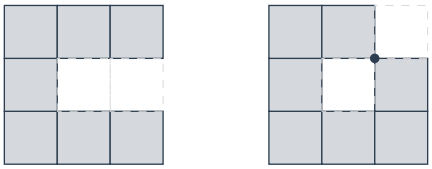


Figure 1: Two polyominoes of seven cells. Left: the empty middle cell is adjacent to an empty cell outside, so there is no hole. Right: the empty middle cell touches the outside only at a corner (marked), so it is a hole. This is the smallest polyomino with a hole, and A001419(7) = 1.

2.2 The Burnside formula

Burnside's lemma says that when a finite group acts on a finite set, the number of orbits is the average, over the group, of the number of elements each group element fixes. The eight symmetries of the square act on the fixed hole-free n-ominoes, and the orbits are the free ones. A symmetry g fixes a fixed polyomino P when $g(P)$ is a translate of P, that is, when P is symmetric under a symmetry of the same kind as g about some point or line. If $g(P) = P + t$, the isometry $x \mapsto g(x) - t$ maps the finite set P onto itself and so fixes its centroid, and an isometry with a fixed point and the same linear part as g is a rotation about that point or a reflection in a line through it, since glide reflections and translations fix no point. Hence

$$8\,a(n) = F(n) + R_2(n) + 2R_4(n) + 2M(n) + 2D(n), \tag{2.1}$$

where the five counts are of fixed hole-free n-ominoes:

- $F(n)$ counts all of them;
- $R_2(n)$ counts those with 180-degree rotational symmetry about some point;
- $R_4(n)$ counts those with 90-degree rotational symmetry about some point, and the 270-degree rotation fixes the same ones;
- $M(n)$ counts those symmetric about some vertical line, and the horizontal reflection fixes as many;
- $D(n)$ counts those symmetric about some line $y = x + c$, and the other diagonal reflection fixes as many.

Each of the five is a single enumeration.

The entry's formula, by Mason, sorts the free polyominoes by their full symmetry group instead. A free polyomino whose symmetry group has order s has $8/s$ fixed copies, and counting the copies that have each symmetry links the two forms:

$$\begin{aligned} M &= 2\,\mathrm{A056879} + 2\,\mathrm{A056880} + \mathrm{A357648}, \\ D &= 2\,\mathrm{A056881} + 2\,\mathrm{A056882} + \mathrm{A357648}, \\ R_2 &= 4\,\mathrm{A056883} + 2\,\mathrm{A056880} + 2\,\mathrm{A056882} + 2\,\mathrm{A357647} + \mathrm{A357648}, \\ R_4 &= 2\,\mathrm{A357647} + \mathrm{A357648}. \end{aligned} \tag{2.2}$$

Section 5.3 uses these as a check.

2.3 Hole-free polyominoes are polygons

Lemma 2.1. *A polyomino has no holes if and only if its boundary is a single self-avoiding polygon, and it is then the set of cells that polygon encloses. Hence $F(n) = A006724(n)$.*

Proof. Call a vertex a *pinch* of P if exactly two of its four cells lie in P and they are diagonally opposite. Suppose P has a pinch v, with cells $c, c' \in P$ at v. A path of adjacent cells of P from c to c', closed up through v, is a closed curve that encloses one of the two empty cells at v. Empty cells adjacent to each other cannot be separated by that curve, so that cell's set of connected empty cells is finite, and P has a hole. So a hole-free polyomino has no pinch, every vertex of its boundary lies on exactly two boundary edges, and the boundary is a disjoint union of self-avoiding polygons. Each of these polygons has the complement of P on one side, all in one component of the complement, and since P is connected, the boundary of each component of its complement is connected, hence a single one of these polygons. So there is one polygon for each component of the complement, the outside and each hole, and a hole-free polyomino has exactly one; it is the set of cells inside that polygon. Conversely, the cells inside a self-avoiding polygon are connected under adjacency, and the cells outside it form a single infinite connected set, since the inside and outside of a polygon are connected open sets and a path in either can be moved off the vertices. □

Jensen and Guttmann (2000, p. 1) make the same identification: polygons enumerated by area are the hole-free polyominoes.

2.4 Forced cores

A symmetric hole-free polyomino must contain certain cells, which the search of Section 3.3 starts from. In both lemmas below, P is a hole-free polyomino, viewed as a closed disk whose boundary C is a simple closed curve (Lemma 2.1).

Lemma 2.2. *If a rotation ρ by 90, 180 or 270 degrees maps P onto itself, its center lies in the interior of P.*

Proof. By Brouwer's fixed-point theorem, ρ has a fixed point in the disk P, and its only fixed point is its center z, so $z \in P$. Suppose $z \in C$. The rotation maps C onto itself and keeps its direction of travel, since it preserves the orientation of the plane and maps the inside of P onto itself. A homeomorphism of a circle that keeps the direction, has finite order and fixes

a point fixes every point, so ρ would fix all of C, which it does not. So z lies in the interior of P. □

So the cells around the center belong to P: one cell, two cells or four cells, for a center at a cell center, an edge midpoint or a vertex.

Lemma 2.3. *If a reflection σ in a line ℓ maps P onto itself, then $\ell \cap P$ is a single segment.*

Proof. The reflection maps C onto itself. It reverses the orientation of the plane and maps the inside of P onto itself, so it reverses the direction of C. A homeomorphism of a circle that reverses the direction has exactly two fixed points, so C meets ℓ in exactly two points. The interior of P is connected and symmetric about ℓ, so it meets ℓ, and $\ell \cap P$ contains a segment of positive length. Each component of $\ell \cap P$ has its endpoints on C, so this segment's two endpoints are the two points of $C \cap \ell$, and it is all of $\ell \cap P$. □

For the three kinds of axis, Lemma 2.3 says:

- on a vertical axis through cell centers, the cells of P on the axis form one unbroken column;
- on a vertical axis between cells, the pairs of cells of P that straddle the axis form one unbroken run;
- on a diagonal axis, the cells of P on the diagonal form one unbroken run, and the corner shared by two consecutive diagonal cells lies inside the segment, hence in the interior of P, so the other two cells at that corner are in P too.

Table 1 lists the resulting cores. For a mirror, the search fixes a spine of k cells on the axis, excludes the other cells on the axis, and grows from the spine, for $k = 1, 2, \ldots$. Placed with its center at the origin, or with its spine starting at height 0, each symmetric hole-free polyomino is then found exactly once.

Count	Center or axis	Core
R_2	cell center	1 cell
R_2	edge midpoint	2 cells, counted for one orientation and doubled
R_2	vertex	4 cells
R_4	cell center	1 cell
R_4	vertex	4 cells
M	through cell centers	k cells in one column
M	between cells	k straddling pairs in one column
D	the line $y = x$	k diagonal cells and the $2(k-1)$ cells at their shared corners

Table 1: The cores from which the search grows each symmetric count.

2.5 M and D as half-size problems

A polyomino symmetric about a vertical axis is determined by the half on one side, and Lemma 2.3 says which halves occur.

Proposition 2.4. *Let $M_{\mathrm{b}}(n)$ and $M_{\mathrm{c}}(n)$ count the fixed hole-free n-ominoes symmetric about a vertical axis between cells and through cell centers, so that $M = M_{\mathrm{b}} + M_{\mathrm{c}}$.*

(a) $M_{\mathrm{b}}(n)$ is 0 for odd n, and for even n it is the number of fixed hole-free polyominoes of $n/2$ cells whose leftmost column is one unbroken run.

(b) $M_{\mathrm{c}}(n)$ is the number of fixed hole-free polyominoes whose leftmost column is one unbroken run of k cells and which have m cells in all, summed over every $k \geq 1$ and $m \geq k$ with $k + 2(m - k) = n$.

Proof. By Lemma 2.3 the axis cuts the disk P along one segment into two disks, so the half on the right of the axis, with the axis cells in case (b), is a hole-free polyomino whose leftmost column is one unbroken run. Conversely, two disks glued along one segment form a disk, so gluing such a half to its mirror image along its leftmost column gives a symmetric hole-free polyomino. □

Both identities agree with the search for $n \leq 22$ and $n \leq 11$ respectively. D has a similar form: it counts the hole-free polyominoes H on or below a diagonal whose cells on the diagonal form one run and whose cells one step below the diagonal each share an edge with that run, with each diagonal cell weighing 1 and every other cell 2. The second condition is needed because a cell one step below the diagonal touches its mirror image at a corner on the diagonal; away from the run, that corner is a pinch, and the pinch encloses a hole. This identity agrees with the search for $n \leq 11$. The results of Section 5 take M and D from the search; Section 8 returns to these identities.

3 Algorithms

3.1 The transfer matrix for F

A transfer matrix counts polygons without listing them, as in Enting (1980) and Jensen and Guttmann (2000). It builds them one vertex at a time and keeps, for each possible *state* of the boundary between the built and the unbuilt part, the number of partial polygons that have it.

Place the polygon in a strip of W cell rows, with vertices (x, y) for $0 \leq y \leq W$. The sweep visits the vertices of each column from bottom to top, and the columns from left to right. After each vertex, the cut between the visited and the unvisited vertices crosses $W + 2$ edges (Figure 2). The state records, for each crossed edge, 0 if the polygon does not use it, and 1 or 2 if the edge carries the lower or the upper end of an arc of the polygon drawn so far. Arcs do not cross, so the 1s and 2s nest like parentheses. These labels and the cut that advances one vertex at a time follow Enting (1980, Sects. 4–5), as does the relabeling when two ends join, below.

- *The step at a vertex.* Of the four edges at a vertex, the polygon uses 0 or 2. Two of them are crossed before the vertex is visited and two after, so the step starts a new arc, extends one, or joins two ends. Joining two lower ends turns the partner of the upper one into a lower end, and joining two upper ends turns the partner of the lower one into

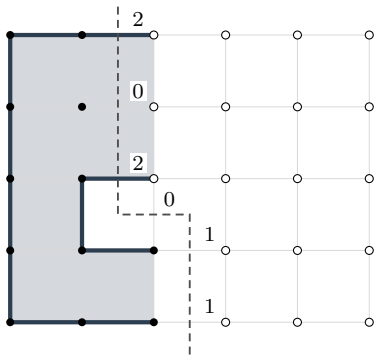


Figure 2: The cut in a strip of $W = 4$ rows, after the sweep has visited vertex $(2, 1)$. Filled dots are visited vertices and open dots unvisited ones. The cut crosses $W + 2 = 6$ edges, labeled $1, 1, 0, 2, 0, 2$ from bottom to top: the outer arc's ends are the first 1 and the last 2, and the inner arc's ends the second 1 and the first 2. The shaded cells are inside the polygon.

an upper end. Joining the two ends of one arc closes the polygon, which is allowed only when no other arc is open.

- *Area.* A cell is inside the polygon when an odd number of used edges lies below it in its column, and the state gives that parity as the sweep passes the cell. Each state carries the number of its partial polygons for each area, up to the target N; only the window of areas the state can still complete within N is stored.

- *Each polygon once.* The empty state is dropped after the first column, so every polygon starts in the first column. A run in a strip of W rows counts a polygon of height $h \le W$ once for each of its $W-h+1$ vertical placements. If T_W is that run's count, the polygons of height exactly h number $T_h - 2T_{h-1} + T_{h-2}$. A 90-degree rotation swaps height and length L, so only polygons with $h \le L$ are needed, counted twice when $h < L$. Since $h + L - 1 \le N$, this gives $h \le \lceil N/2 \rceil$. The runs record each polygon's length, so this selection is made after the second difference.

- *Pruning.* Every column of a polygon contains a cell, and the run in a strip of W rows needs only polygons with $L \ge W$. So a state after column c is dropped once its least area exceeds $N - (W - 1 - c)$. The second difference above stays exact because it is taken at each length L separately: for $h \le L$ it needs strips h, $h - 1$ and $h - 2$ only at lengths $L \ge h$, and the strip of W rows keeps every polygon of length at least W.

- *The mirror merge.* The strip is symmetric from top to bottom, so at a column boundary a state and its mirror image, with the positions reversed and the labels 1 and 2 swapped, have the same completions. The two are merged into one. Barequet and Ben-Shachar (2026, Sect. 4.3) merge mirror-image states in odd-height boxes on the rotated lattice and note that the merge applies to the unrotated algorithm too. Here it runs on the unrotated strip for every W, and the test for R_2 of Section 3.2 stays exact on the merged states. At $N = 24$ it removes 44% of the states at column boundaries.

One run to N gives $F(1), \ldots, F(N)$. It truncates every count at area N, so it cannot later be extended to $N + 1$: each N chosen costs a full run. A state fits in a 64-bit word at 2 bits

per crossed edge when $W + 2 \leq 32$, that is, for $N \leq 60$. The unit of work is the *state-step*, one state processed at one vertex. State-steps are exact and the same on every machine, and Section 6 measures the work in them.

3.2 R_2 from the same transfer matrix

Shirakawa (2025, Sect. 6.1) suggests, as future work, a transfer matrix for the point-symmetric polyominoes with a central core, which take most of the time of that enumeration. For hole-free polyominoes, this section computes R_2 inside the transfer matrix for F, with no sweep of its own.

A polygon with 180-degree rotational symmetry is its left half together with the half-turned copy of that half. The transfer matrix for F already carries every left half that R_2 needs, and any valid pruning bound keeps them, since the half-turned copy is one possible completion. So R_2 needs only a test on the states that F already has, made where the sweep reaches the polygon's center. The center is placed on the middle line of the strip, at height $W/2$, so the polygon's height h has the parity of W. The sweep counts only polygons with $h \leq L$, which already uses the 90-degree rotation, so all four kinds of center are tested directly; neither kind of edge midpoint can stand in for the other.

A state passes the test if its occupancy pattern, which edges are used, is symmetric under the half-turn ρ, no used edge touches the center, and its arcs together with their half-turned copies form a single loop. The occupancy test is a bit reversal of the state word, and only the few states that pass it reach the loop test. Column c of cells lies between vertex columns c and $c + 1$, and a partial polygon's area a counts its inside cells whose lower left corner the sweep has visited. The swept cells and their images under ρ cover the polygon's columns and overlap in the cells named below, so the polygon's area is $2a$ less the inside cells among them:

- *Odd* $L = 2c + 1$. The center lies in column c of cells, and the test runs after the last vertex of vertex column c, where the cut crosses the $W + 1$ horizontal edges of that column. The swept cells are columns 0 to c, and ρ maps column c onto itself, so the area is $2a - i$, where i counts the inside cells of column c.
  - *Cell center* (odd h): the center of the cell in row $(W - 1)/2$ of column c, which no edge of the cut passes through.
  - *Midpoint of a horizontal edge* (even h): the midpoint of the edge of column c at height $W/2$, which must be unused.
- *Even* $L = 2c$. The center lies on vertex column c, and the test runs after the vertex in row $\lceil W/2 \rceil - 1$ of that column. The swept cells are columns 0 to $c - 1$ and the cells of column c below row $\lceil W/2 \rceil$, and ρ maps the latter onto cells of column $c - 1$, so the area is $2a - 2i$, where i counts the inside cells of column c below row $\lceil W/2 \rceil$.
  - *Vertex* (even h): the vertex $(c, W/2)$. The cut crosses two of its edges, the vertical edge below it and the horizontal edge to its left, and both must be unused; a symmetric polygon through the center would use one of them.
  - *Midpoint of a vertical edge* (odd h): the midpoint of the edge from $(c, (W - 1)/2)$ to $(c, (W + 1)/2)$, which the cut crosses and which must be unused.

Each polygon with 180-degree symmetry, height h and length $L \geq h$ is counted once, at its center, in each strip of W rows with $h \leq W \leq L$ and $W \equiv h \pmod 2$. Its first column is column 0 and its center is at height $W/2$, so its placement in that strip is unique, and a polygon has one center of symmetry, the center of its bounding box, so only the test at that center can count it. There the sweep carries its swept part once, as one partial polygon of one state, since the pruning keeps every polygon of length at least W. The state passes the test: its occupancy is symmetric because the polygon is, no used edge touches the center because Lemma 2.2 puts the center in the interior, and the arcs and their copies form the polygon, a single loop. Conversely, if a state passes the test, the vertices the sweep has visited and their images under ρ are disjoint, so every vertex of the partial polygon and its half-turned copy lies on two of their edges, and the loop test makes the union one closed curve: a self-avoiding polygon symmetric about the center, of length L and of the area above. So at each length $L \geq W$, the strip of W rows counts the polygons of heights $W, W-2, \ldots$, each once, and $R_W - R_{W-2}$ at that length, where R_W is that strip's count, gives those of height exactly W. As for F, a polygon with $h < L$ is counted twice, for its 90-degree rotation, which is also symmetric under the half-turn.

The mirror merge of Section 3.1 keeps the test exact. The top-bottom mirror σ of the strip commutes with the half-turn about any point of the middle line, so it maps the polygons symmetric about a center onto the polygons symmetric about the same center, with the same areas. The merge replaces some partial polygons at a column boundary by their mirror images, and σ matches the symmetric completions of a partial polygon one to one with those of its mirror image, so the count that passes each later test is unchanged. The program's tests compare R_2 from the sweep with R_2 from the search at $N = 20$, with the merge and without it.

3.3 The search for the symmetric counts

Redelmeier's algorithm (Redelmeier 1981) lists every connected set of cells that contains a given start cell, each exactly once. Started from a connected set of cells in place of one cell, the same recursion lists every connected set that contains it. It keeps a list of untried neighbors of the current set; each untried cell is either added, and its new neighbors join the list, or excluded for the rest of that branch. Run on the orbits of a symmetry g in place of single cells, starting from a core of Table 1, it lists the polyominoes that contain the core and are symmetric under g.

Holes are detected with the Euler characteristic $\chi = V - E + F$ of the union of the closed cells, where V, E and F count its distinct corners, edges and cells. For a connected polyomino, χ is 1 minus the number of holes. Adding a cell changes χ by the number of its corners not yet present plus the number of its adjacent cells already present, minus 3. The set grows from a connected core, so it stays connected, and it is counted when it reaches n cells with $\chi = 1$.

The search computes R_2 too, as a check on the transfer matrix. A deep search splits into units: at a fixed depth, unit i of p continues only every p-th branch, the split that Shirakawa (2025, Sect. 2.4.2) uses across threads. The units depend only on the search, not on the machine, and the threads share them.

3.4 Exact counts from residues

The transfer matrix keeps its counts in 64-bit words and works modulo one number per run. Two runs, modulo 2^{64} and modulo the prime $2^{61}-1$, give each count modulo their product by the Chinese remainder theorem. Enting (1980, p. 3720) also counted modulo several primes and rebuilt the integers from the residues. The second difference subtracts, which modular arithmetic handles as it is. $F(n)$ and $R_2(n)$ are at most the number of fixed polyominoes, A001168(n), which is below $2^{64}(2^{61}-1)$ for every $n \le 65$, so the two runs give exact counts for every n the 64-bit state allows.

A sweep can also run with 16-bit counts, one run per 16-bit modulus, in about a third of the memory per state. Section 5.2 uses the moduli 2^{16}, 65521, 65519, 65497 and 65479, whose product, about $2^{80.0}$, exceeds A001168(42) $\approx 2^{77.9}$.

3.5 Independent pieces

The computation splits along several axes into pieces that never interact, and the pieces add back up. Table 2 lists the splits.

Split	Pieces	Recombined by	Checked in each run	Checked in the tests
symmetry	F, R_2, R_4, M, D	Equation (2.1)	sum is a multiple of 8	all five against a brute force at small n
modulus	residues modulo each modulus	Chinese remainder theorem	none of its own	exact F against A006724 to $n = 20$; 16-bit counts against 64-bit
strip height	strips $W = 1, \ldots, \lceil n/2 \rceil$	second differences and sum	none of its own	F against A006724 to $n = 25$
occupancy	groups of states within a column	disjoint union	none of its own	same results in memory and on disk
time	the columns of a strip	resuming from a checkpoint	none of its own	same output after kills at random times
search branch	branches at a fixed depth	sum	R_2 by sweep equals R_2 by search	same unit files on 2 and 14 threads

Table 2: The splits of the computation into independent pieces. A run checks only the sums of its own output; the program's test suite checks the splits at small n.

- *Occupancy.* While the sweep processes one half of a column, states that differ in the occupancy pattern of the other half never combine. This is Jensen's split for polygons (Jensen 2003a, Sect. 2.2), which Barequet and Ben-Shachar (2026, Sect. 4.4) describe for polyominoes, from Jensen (2003b). The groups run on separate threads. Under a memory limit they also let the sweep run out of core: once a step would pass the limit,

the states are written to buckets on disk by group, and each bucket is processed in memory. The results and the state-steps are the same as in memory.

- *Time.* The sweep saves its states at column boundaries and each finished strip's tables, and the search saves each finished unit. A stopped run resumes from them and produces the same output.
- *Modulus and strip height.* The runs keep each finished strip's tables for each modulus, and the final counts are rebuilt from them. Separate 16-bit runs trade time for memory.
- *Search branch.* The units of Section 3.3 run on separate threads and are summed.

In a run, four of the splits have no check of their own: a wrong residue, strip table, occupancy group or resume at the run's n shows up only in the sum of Equation (2.1). An error e in F or R_2 escapes that check only when $8 \mid e$, and an error in R_4, M or D only when $4 \mid e$, and the check does not say which count is wrong. The sums over the subgroups of the symmetry group narrow it down (Section 5.2).

4 The program

The program is written in Rust, a compiled language without a garbage collector, and ported from a slower Python program that its tests compare it with (Appendix A). It runs the sweep and then the search on one machine, on the threads of the rayon library. Its command `components 46` computes the five counts and $a(n)$ for every $n \leq 46$.

4.1 The sweep's states

- *Layers.* The states at one point of the sweep form a *layer*. A state is a 64-bit word, 2 bits per crossed edge (Section 3.1), and carries its counts for a window of areas: in column c, its i-th count is the number of its partial polygons of area $c + i$, and the window ends at the largest area that can still close in a polygon of length at least W.
- *Shards.* A layer of 16,384 states or more is split into four *shards* per thread, at most 1,024, by a multiplicative hash of the state word. A shard holds a hash table from each state word to its row, the state words, and the counts, one fixed-width row per state.
- *The step in two passes.* A step at a vertex runs in two parallel passes: each shard maps its states to their successors, at most two each, and routes every successor to the shard of its new word; then each shard builds its next table from what was routed to it, adding the counts of successors that share a word.
- *Column boundaries.* At the end of a column the states shift into the next column's slots, their windows of areas move with them, and each state merges with its mirror image (Section 3.1).
- *Counts.* The sweep only adds counts and moves them between areas, so each modulus needs only addition and subtraction: wrapping arithmetic modulo 2^{64}, and one comparison and correction after each sum or difference modulo $2^{61} - 1$ and the 16-bit primes. Garner's algorithm rebuilds the exact values from the residues (Section 3.4).

4.2 Memory and disk

- *A memory limit.* Under `-mem-limit` the program keeps its heap below the limit less 64 MiB. It bounds the bytes of each step before running it, with every table sized for exactly its states, and it maps blocks of 64 KiB or more straight from the operating system, so the memory a step frees leaves the process at once.
- *Bucket files.* A step that would pass the limit moves its column's states to disk, by the occupancy split of Section 3.5. The column runs in two windows of rows. Its states are grouped by the occupancy of the slots outside the window, which the window's steps leave unchanged, and the groups are spread over up to 4,096 bucket files by a hash of that occupancy. Each bucket is loaded and stepped in memory. A bucket too big for the room splits: its window halves, or with one row left, its groups spread over more buckets.
- *Checkpoints.* With `-checkpoint-dir` the program saves each finished strip's tables, the strip in progress at a column boundary once the checkpoint interval has passed since the last save, and each finished unit of the search. Each file is written under a temporary name, flushed, renamed and covered by a checksum, so a stop in mid-write leaves the last good file in place, and a damaged file is reported and left unused. A settings file ties the directory to the command and to the commit of the build, and the program refuses a directory made by another command or build. A saved layer loads back as shards on any number of threads.

4.3 The search

- *Orbits.* The search lists the orbits of one symmetry on the cells within distance $n + 1$ of the origin, each with its cells and its neighboring orbits, and adds an orbit one cell at a time to keep the Euler characteristic of Section 3.3.
- *Units.* A run more than 11 orbits deep splits at depth 8 into 4^{d-11} parts, at most 1,024, where d is its depth. The split depends on the run alone, so the units are the same on every machine and number of threads, and a run can resume any of them anywhere. The threads share the units.
- *R_2 by search.* The search computes R_2 only when asked, with `-r2 search` or `-r2 both`; `both` also requires it to equal R_2 from the sweep.

4.4 Tests

- *Known values.* The counts against the data sections of the OEIS entries, against the brute force of Section 5.3, and the state-steps and peak states against the Python program's; with the mirror merge turned off they are equal.
- *Stops.* Sweeps stopped between strips and resumed against whole ones; runs killed after a column checkpoint, inside column writes, from memory to disk and back, and in the search, then run again; damaged files and mismatched settings; and the unit files on 2 and 14 threads, which must be the same.

- *Disk.* The bucket files, and sweeps on disk against the sweep in memory, with an allocator that counts the heap to check the limit.

The kills at random times, at $n = 30$ to 34, take minutes and run only on request.

5 Results

5.1 New terms

Table 3 gives the new terms of A000104 and A001419, and Table 4 the five counts of Equation (2.1) at the same n; its F column gives the new terms of A006724 and the corrected value at $n = 42$. The values of A000105 come from its b-file. The terms at $n = 41$ and 42 come from runs on a laptop with an Apple M4 Pro (14 cores, 48 GB), and those at $n = 43$ to 46 from runs on Amazon EC2, the compute service of Amazon Web Services, each run on one arm64 machine, with a Graviton4 or Graviton3 processor. At $n = 40$, A001419(40) = 1073051863257197250101 is still below $a(40) = 111421203340763298936 6$, so $n = 41$ is the first n at which polyominoes with holes outnumber those without.

n	A000104, without holes	A001419, with holes
41	4320025486567288556616	4352712104645074863609
42	16758946477869251303088	17649230129410250476504
43	65048612954666785866679	71537300655036412731948
44	252609689407501831000067	289863312298856051732003
45	981460883367208541879111	1174139207740115687375304
46	3815030495829137292573170	4754689837467697275861435

Table 3: New terms of A000104 and A001419.

n	$F(n)$, A006724	$R_2(n)$	$R_4(n)$	$M(n)$	$D(n)$
41	34560203892113934050327	153844599679	24183	98492082007	36772795271
42	134071571821918373469990	549263328294	0	212538461660	30648351550
43	520388903635712449766902	595530895538	0	373961038792	139192096704
44	2020877515256041449179842	2126972080014	71610	807215423974	115897874756
45	7851687066931459562295813	2307884663989	86479	1422419327665	528024622399
46	30520243966617831715685344	8245543032236	0	3071271433752	439269500138

Table 4: The five counts of Equation (2.1). $F(41)$ agrees with the A006724 b-file; $F(42)$ corrects it, and $F(43)$ to $F(46)$ are new.

5.2 The error in A006724 at n = 42

The A006724 b-file gives $F(42) = 134071571821918373415776$. With this value and the searched R_2, R_4, M and D, the sum in Equation (2.1) is 2 (mod 8), so it is not $8\,a(42)$. Burnside's lemma applies to every subgroup of the symmetry group as well, and each subgroup gives a sum that must be a multiple of the subgroup's order. Table 5 shows these sums at $n = 42$.

Subgroup	Sum	Residue at $n = 42$
180-degree rotation	$F + R_2 \pmod 2$	0
vertical reflection	$F + M \pmod 2$	0
diagonal reflection	$F + D \pmod 2$	0
rotations	$F + R_2 + 2R_4 \pmod 4$	2
both axis-parallel reflections	$F + R_2 + 2M \pmod 4$	2
both diagonal reflections	$F + R_2 + 2D \pmod 4$	2
all eight symmetries	$F + R_2 + 2R_4 + 2M + 2D \pmod 8$	2

Table 5: The subgroup sums at $n = 42$, with $F(42)$ from the A006724 b-file.

If only one count is wrong, it is F or R_2: an error in R_4, M or D alone would leave two of the three sums modulo 4 at 0. Its error is then 2 (mod 4). These sums always pair F with R_2, so they cannot say which of the two is wrong.

A transfer-matrix run to $N = 42$ decides it, since it computes F and R_2 independently of both the b-file and the search. The exact run needs more memory than the 48 GB laptop it ran on, so each run counted modulo one of the five 16-bit moduli of Section 3.4, and the Chinese remainder theorem combined them into exact values. Its F equals the b-file for every $n \le 41$, and its R_2 equals the search for every $n \le 42$. At $n = 42$ it gives

$$F(42) = 134071571821918373469990,$$

which is 54,214 more than the b-file. Since $54214 \equiv 6 \pmod 8$, this value makes the sum in Equation (2.1) a multiple of 8, and it gives $a(42)$. The later exact runs to $n = 43$ to 46, with the two 64-bit moduli, give the same $F(42)$.

The b-file agrees line for line with Jensen's series file linked from the entry (Jensen 2016), so the error is in the series itself. Guttmann et al. (2000), who extended the series to 42 terms, do not print its coefficients; they give Jensen's web page as the place to obtain it, and the series file is the archived copy from that page.

5.3 Checks

The terms pass the following checks.

- For $n \le 11$, a brute force that lists every fixed polyomino and reduces each to a canonical form under the eight symmetries agrees with F, R_2, R_4, M, D and a.
- For $n \le 40$, $a(n)$ agrees with the A000104 b-file, and R_2, R_4, M and D agree with Equation (2.2), using the data sections of the exact-symmetry entries, which reach $n = 40$.
- For $n \le 41$, $F(n)$ agrees with the A006724 b-file.
- For every $n \le 46$, R_2 from the transfer matrix equals R_2 from the search, and every subgroup sum of Table 5 is 0.

- The runs to $n = 43$, 44, 45 and 46 each reproduce the rows of the run before. The run to $n = 45$ was stopped nine times and resumed from its checkpoints, and its output passes the same checks.
- The program's tests check its state-step counts against an independent Python implementation of the same transfer matrix, and the out-of-core sweep against the sweep in memory, for every strip at $N = 20$ to 28.

Above $n = 41$, the checks that use none of the sweep's code reach F only modulo 8. R_2 from the search checks the sweep's R_2 and leaves its F unchecked. The Burnside sum and the subgroup sums catch an error in F only when it is not a multiple of 8. For $n \leq 41$, F agrees with Jensen's series, from a separate program; a defect that only the widest strips or the out-of-core sweep reach at larger N would not show there. The runs that reproduce each other's rows differ in their truncation N and in which strips stream through disk, and they share the rest of the code, so a defect common to all of them would not show either. At $n = 42$ the 16-bit and the 64-bit runs agree; they differ in the width of their counts and share the sweep. So $F(43)$ to $F(46)$, and with them $a(43)$ to $a(46)$, rest on one implementation of the sweep.

6 Work and memory

The sweep's work is measured in state-steps, and its memory in *peak states*, the largest number of states held at any one vertex. Both are exact and the same on every machine. The search is measured in *vCPU-seconds*, seconds on one of a machine's virtual processors, its vCPUs. Table 6 lists them for the runs from $n = 42$ to 46.

n	State-steps	Ratio	Peak states	Ratio	Widest strip's share
42	104.9 billion		122.8 million		50%
43	239.6 billion	2.284	246.9 million	2.011	48%
44	288.7 billion	1.205	307.0 million	1.243	50%
45	656.5 billion	2.274	617.9 million	2.013	48%
46	791.4 billion	1.205	768.2 million	1.243	49%

Table 6: The sweep's work and memory, for one modulus. Each ratio is to the row above.

- *The growth rates.* Fitted to the laptop's runs from $n = 32$ to 42, the work grows by 1.669 a term and the peak states by 1.578. Over two terms the later runs agree: from $n = 42$ to 44 and from 44 to 46, the work grows by 1.659 and 1.656 a term and the peak states by 1.581 and 1.582.
- *Odd and even n.* The widest strip, of $\lceil n/2 \rceil$ rows, does about half the work and holds the whole memory peak. Each odd n adds a new widest strip, and the even n after it reuses that strip. So the work grows 2.28 and 2.27 times from $n = 42$ to 43 and from 44 to 45, and 1.20 and 1.21 times at the even steps after them; the peak grows 2.01 times at each odd n and 1.24 times at each even n. The fit carries the odd steps as factors of 1.36 for the work and 1.27 for the peak: $a(45)$ measured 1.34 and 1.28, and $a(44)$ and $a(46)$ came within 3% of the fit with no factor.

- *Memory per peak state.* The whole run's peak memory over its peak states was 613 bytes at $n = 43$ and 647 bytes at $n = 44$, with 64-bit counts. With 16-bit counts it was 220 bytes at $n = 42$.
- *The search* took 69,344 vCPU-seconds at $n = 43$, 144,685 at $n = 44$ and 578,755 at $n = 46$, and its fitted growth over $n = 32$ to 42 is 1.982 a term. It needs about 14 MB of memory and keeps every vCPU busy.
- *The search's growth on one processor* is below the fit. On Graviton4 it grew about 1.84 a term from $n = 43$ to 45. The same run to $n = 36$ on two machines of the same size that differ only in the processor took 1.24 times as long for the search on Graviton3, and 1.04 times as long for the sweep. The run to $n = 46$ ran on Graviton3, so on Graviton4 its 578,755 vCPU-seconds would have been about 467,000, a growth of about 1.89 a term from $n = 43$. The fit covers the whole search, with the parity cost below and R_2's three jobs, which only check the sweep.
- *The sweep leaves vCPUs idle,* limited by its own parallelism: 19 to 24 of the 32 vCPUs were busy at $n = 43$, 24 to 39 of the 64 at $n = 44$ and 45, and 18 to 24 of the 48 at $n = 46$.
- *The search by job.* Its unit files time each job (Table 7). The shares moved by at most 1.7 points from $n = 45$ to 46. R_2's three jobs take 54 to 56% of the search, and they serve only as a check on the sweep's R_2.
- *A cost of parity.* The search grows a set while it has fewer than n cells, so at $n = 46$ every 45-cell set in a job whose sizes are all odd scans its untried orbits, none of which fits, since each adds at least 2 cells; at $n = 45$ those sets are leaves. The jobs whose counts are the same at both n show it: on the laptop, which isolates it from any change of machine, their parts took 1.93 times as long at $n = 38$ as at $n = 37$.

Job	$n = 45$	Share	$n = 46$	Share	Ratio
R_2, cell center	16.4	26.9%	40.8	26.5%	2.49
R_2, edge midpoint	11.8	19.3%	32.3	21.0%	2.75
R_2, vertex	4.8	7.9%	12.4	8.1%	2.57
M, axis through cells	17.8	29.3%	43.7	28.4%	2.45
M, axis between cells	2.7	4.5%	7.0	4.5%	2.55
D	7.3	12.0%	17.5	11.4%	2.41
All the parts	60.8		153.8		2.53

Table 7: The search's vCPU-hours by job, from the unit files of the runs to $n = 45$ and 46. A part's time is the median gap between consecutive unit files in its run, on one thread. R_4's jobs are too small to time. The run to $n = 46$ ran on Graviton3 and the run to $n = 45$ on Graviton4, so each ratio includes the search's 1.24 between the two processors.

7 The constant and the growth rate

Write the sweep's work as $c\,g^n$. Changes to the code, the state encoding or the machines multiply the constant c; changes to the algorithm's pruning change the growth rate g.

- *The constant.* A change to c scales the work of every run by the same factor. The work rises about 2.75 times from one even n to the next (Section 6), so a speedup below 2.75 reaches no further even target in a run of the same length.
- *The growth rate.* A lower g compounds with every term. The rates 1.53 and 1.44 are the growth of the signatures processed from $n = 30$ to 40 in Table 2 of Barequet and Ben-Shachar (2026), for their implementations of the unrotated and the rotated transfer matrices for fixed polyominoes. They are illustrative targets: nothing yet shows that a sweep for polygons reaches them.

8 Further work

- *Lowering the sweep's growth rate.* The pruning bound of Section 3.1 counts only the columns a state needs to reach the strip's length. An exact least completion area for each state, from a small dynamic program over its arcs, the untouched borders of the strip and the remaining columns, would prune more. The 45-degree rotated sweep of Barequet and Ben-Shachar (2026) lowered the growth of the polyomino transfer matrix from 1.53 to 1.44, which they attribute to pruning: on the rotated lattice, closing a vertical gap of k cells takes $2k + 1$ cells, against $k + 2$ on the unrotated one (Barequet and Ben-Shachar 2026, Sect. 5). For polygons, the R_2 test of Section 3.2 would have to be derived again on the rotated lattice, and the rates 1.53 and 1.44 remain illustrative targets (Section 7).
- *The search's growth rate.* The search grows faster than the sweep: from $n = 43$ to 45, on Graviton4, its vCPU-seconds grew 3.38 times, about 1.84 a term, and the sweep's state-steps 2.74 times, about 1.66 a term; the fit to the laptop's runs, 1.982, is higher (Section 6). On each machine the search took about a third of the sweeps' time: 2,167 s against 6,358 s at $n = 43$, and 2,261 s against 5,929 s at $n = 44$. Proposition 2.4 and its analogue for D turn M and D into transfer matrices of half the size, as Shirakawa (2025) does for the mirror classes of all polyominoes. R_2 already comes from the sweep, and R_4 is small: Shirakawa (2025, Table 2) reports well under an hour for the 90-degree classes of all polyominoes to $n = 60$.
- *The search's constant.* Two changes cut the search's time at every n. Stopping each job one level early where no orbit can fit removes the parity cost, which on the laptop nearly doubled the time of the jobs it affects. And R_2's three jobs, 54 to 56% of the search, only check the sweep's R_2; the program skips them unless that check is asked for, so a run can trade the check for half the search's time.
- *The exact-symmetry sequences.* The OEIS also counts hole-free polyominoes by their full symmetry group, in the seven sequences of Equation (2.2), and four of them, A056879, A056881, A056882 and A056883, end at $n = 40$. Our counts already extend one. The first line of Equation (2.2) gives $\mathrm{A056879}(n) = (M(n) - 2\,\mathrm{A056880}(n) - \mathrm{A357648}(n))/2$, and the published A056880 and A357648 reach $n = 52$ and $n = 77$, so A056879 at $n = 41$ to 46 is 49245943996, 106269158340, 186980333252, 403607573001, 711209321148 and 1535635466062. The other three cannot be separated with D and R_2 alone. They need one more count, of the polyominoes symmetric under both diagonal reflections, which is a search over orbits of four cells, small next to the searches above.

- *Measuring progress.* The growth rate is measured in state-steps, which are exact and the same on every machine; a fit over $n = 28$ to 36 takes minutes on a laptop. A change is kept only if F still matches A006724, R_2 from the sweep still equals R_2 from the search, and every subgroup sum is still 0. The constant is measured as time per state-step at a fixed n on one machine. The two are kept apart: a change that multiplies c by $\alpha < 1$ and g by $\beta > 1$ wins below $n = \log(1/\alpha)/\log\beta$ and loses above it, so a single timing at small n cannot rank it.

A Data and programs

The programs, data and LaTeX source are at `https://github.com/charlesreid1/A000104`.

- `tex/`: the LaTeX source of this manuscript and its companion note.
- `data/terms-46.txt`: the new terms of A000104, A001419 and A006724, and the corrected A006724(42).
- `data/components-46.txt`: n, F, R_2, R_4, M, D and $a(n)$ for $n \le 46$, from the run to $n = 46$; `data/residues-42*.txt`: the five 16-bit sweeps at $n = 42$; and the b-files used in the checks.
- `rust/`: the program; `python/a000104.py`: the slower Python program it was ported from, which the tests compare it with.
- `checks.py`: the checks of Section 5.3; `measurements.md`: every measured run.

From the repository's top directory, these commands build the program, reproduce $a(0)$ to $a(28)$ against the b-files and the five counts to $n = 36$, and rerun the checks of Section 5:

```
cd rust && cargo build --release && cd ..
rust/target/release/a000104 terms 28 --check data
rust/target/release/a000104 components 36 --r2 both
python3 checks.py crt data/components-42.txt data/residues-42*.txt
python3 checks.py subgroups data/components-46.txt
python3 checks.py mason data/components-46.txt
```

The run to $n = 36$ takes 135.8 s on the laptop of Section 5, and its rows match the first rows of `data/components-46.txt`. The run to $n = 44$, `components 44 -r2 both`, needs about 185 GiB of memory; it took 2 h 16 min on the 64 vCPUs of an r8g.16xlarge.

References

Barequet, Gill and Gil Ben-Shachar (2026). "Counting Polyominoes, Revisited". In: *Algorithmica* 88, 51. DOI: 10.1007/s00453-026-01392-6. arXiv: 2610.10632.

Conway, A. R. (1995). "Enumerating 2D percolation series by the finite-lattice method: theory". In: *Journal of Physics A: Mathematical and General* 28.2, pp. 335–349. DOI: 10.1088/0305-4470/28/2/011.

Enting, I. G. (1980). "Generating functions for enumerating self-avoiding rings on the square lattice". In: *Journal of Physics A: Mathematical and General* 13.12, pp. 3713–3722. DOI: 10.1088/0305-4470/13/12/021.

Guttmann, Anthony J. et al. (2000). "Punctured polygons and polyominoes on the square lattice". In: *Journal of Physics A: Mathematical and General* 33, p. 1735. arXiv: cond-mat/0003441.

Jensen, Iwan (2003a). "A parallel algorithm for the enumeration of self-avoiding polygons on the square lattice". In: *Journal of Physics A: Mathematical and General* 36.21, pp. 5731–5745. DOI: 10.1088/0305-4470/36/21/304. arXiv: cond-mat/0301468.

– (2003b). "Counting polyominoes: a parallel implementation for cluster computing". In: *Computational Science, ICCS 2003*. Vol. 2659. Lecture Notes in Computer Science. Springer, pp. 203–212. DOI: 10.1007/3-540-44863-2_21.

– (2016). *square.area.ser*. Square lattice polygons by area, the "More terms" link of the OEIS entry A006724; the Internet Archive copy of January 2, 2016. URL: https://web.archive.org/web/20160102114247/http://www.ms.unimelb.edu.au/~iwan/polygons/series/square.area.ser (visited on 11/01/2026).

Jensen, Iwan and Anthony J. Guttmann (2000). "Statistics of lattice animals (polyominoes) and polygons". In: *Journal of Physics A: Mathematical and General* 33.29, p. L257. DOI: 10.1088/0305-4470/33/29/102. arXiv: cond-mat/0007238.

Mason, John (Oct. 11, 2022). *Counting unholey polyominoes*. Description of counting programs, linked from A000104. URL: https://oeis.org/A000104/a000104_2.pdf (visited on 11/01/2026).

Redelmeier, D. Hugh (1981). "Counting polyominoes: yet another attack". In: *Discrete Mathematics* 36.2, pp. 191–203. DOI: 10.1016/0012-365X(81)90237-5.

Shirakawa, Toshihiro (2025). "Enumeration of Polyominoes up to Size $N = 59$". In: arXiv: 2510.22446 [math.CO].

OEIS sequences

A000104 *Number of n-celled free polyominoes without holes*. The On-Line Encyclopedia of Integer Sequences. URL: https://oeis.org/A000104 (visited on 11/01/2026) (cited on page 1).

A000105 *Number of free polyominoes (or square animals) with n cells*. The On-Line Encyclopedia of Integer Sequences. URL: https://oeis.org/A000105 (visited on 11/01/2026) (cited on page 1).

A001168 *Number of fixed polyominoes with n cells*. The On-Line Encyclopedia of Integer Sequences. URL: https://oeis.org/A001168 (visited on 11/01/2026) (cited on pages 2, 10).

A001419 *Number of n-celled polyominoes with holes*. The On-Line Encyclopedia of Integer Sequences. URL: https://oeis.org/A001419 (visited on 11/01/2026) (cited on page 1).

A006724 *Number of fixed n-celled self-avoiding polygons on square lattice*. The On-Line Encyclopedia of Integer Sequences. URL: https://oeis.org/A006724 (visited on 11/01/2026) (cited on page 1).

A056879 *Number of n-celled axially symmetric polyominoes without holes*. The On-Line Encyclopedia of Integer Sequences. URL: https://oeis.org/A056879 (visited on 11/01/2026) (cited on page 4).

A056880 *Number of n-celled polyominoes without holes, symmetric about axis 2*. The On-Line Encyclopedia of Integer Sequences. URL: https://oeis.org/A056880 (visited on 11/01/2026) (cited on page 4).

A056881 *Number of n-celled diagonally symmetric polyominoes without holes*. The On-Line Encyclopedia of Integer Sequences. URL: https://oeis.org/A056881 (visited on 11/01/2026) (cited on page 4).

A056882 *Number of n-celled free polyominoes that have no holes and are symmetric about two diagonal axes (and therefore centrally symmetric) but have no other symmetries*. The On-Line Encyclopedia of Integer Sequences. URL: https://oeis.org/A056882 (visited on 11/01/2026) (cited on page 4).

A056883 *Number of n-celled rotationally symmetric polyominoes without holes*. The On-Line Encyclopedia of Integer Sequences. URL: https://oeis.org/A056883 (visited on 11/01/2026) (cited on page 4).

A357647 *a(n) is the number of free unholey polyominoes of n cells with 90-degree rotational symmetry and no other*. The On-Line Encyclopedia of Integer Sequences. URL: https://oeis.org/A357647 (visited on 11/01/2026) (cited on page 4).

A357648 *Number of polyominoes with n cells that have the symmetry group D_8 and are without holes*. The On-Line Encyclopedia of Integer Sequences. URL: https://oeis.org/A357648 (visited on 11/01/2026) (cited on page 4).